

INF 662 · DESARROLLO DE APLICACIONES MÓVILES

Guía de Laboratorio N.º 7

Ejemplo integrador: registro de estudiantes

MATERIA INF 662 — Desarrollo de Aplicaciones Móviles**SEMESTRE** 6.º · Mención Ing. de Software**TEMA** Ejemplo resuelto de consolidación**DURACIÓN** 1 sesión**CAPÍTULOS** Consolida Cap. 1–3 (Guías 1 a 5)**MODALIDAD** Laboratorio · individual**DOCENTE** M. Sc. Lic. Huáscar Fedor Gonzales Guzmán**REQUISITO** Guías de Laboratorio N.º 1 a N.º 5**NOTA · Segundo ejemplo integrador**

Como la Guía 6, este es un **ejemplo resuelto** que reúne lo aprendido, ahora con foco en **formularios de varios campos**. Construirás un **registro de estudiantes**: un formulario validado (nombre, edad y carrera) que agrega cada estudiante a una lista y la muestra en pantalla.

01 Objetivos

Objetivo general. Integrar el manejo de formularios, validación de varios campos y estado local en una aplicación de registro que administra una lista de estudiantes en memoria.

Objetivos específicos:

- Definir un modelo de datos propio (Estudiante) con varios campos.
- Validar varios `TextFormField` y un `DropDownButtonFormField` dentro de un `Form`.
- Agregar y eliminar elementos de una lista con `setState`.
- Limpiar el formulario tras un registro válido con `reset()` y `clear()`.
- Presentar los datos con `ListView.builder`, `ListTile` y `CircleAvatar`.

02 Análisis del problema

La app tendrá una pantalla dividida en dos zonas: arriba, el formulario de registro; abajo, la lista de estudiantes ya registrados con un contador y opción de borrar. Los campos y sus reglas:

EJEMPLO · Campos y validación**Nombre:** texto obligatorio, mínimo 3 caracteres.**Edad:** número obligatorio entre 15 y 99.**Carrera:** selección obligatoria de una lista fija (dropdown).

03 Requisitos previos

Requisito indispensable. Haber completado las Guías N.º 1 a N.º 5 (en especial la 5, de formularios y validación).

Conceptos que se reutilizan:

Form + validate()	TextFormField	DropDownButtonFormField
setState + listas	Modelo de datos	ListView.builder

04 Desarrollo · Parte A — Modelo y estado

1 Definir el modelo Estudiante

1. Crea el proyecto `registro_app`. En `lib/main.dart`, define la clase de datos y la lista fija de carreras:

```
LIB/MAIN.DART · DART
class Estudiante {
  final String nombre;
  final int edad;
  final String carrera;

  const Estudiante(this.nombre, this.edad, this.carrera);
}

const carreras = <String>[
  'Ing. Informatica',
  'Ing. de Sistemas',
  'Ing. Civil',
  'Ing. Comercial',
];
```

2 Preparar la pantalla con estado

1. Crea un `StatefulWidget` con la clave del formulario, los controladores de nombre y edad, la carrera seleccionada y la lista de estudiantes:

```
LIB/MAIN.DART · DART
class PantallaRegistro extends StatefulWidget {
  const PantallaRegistro({super.key});

  @override
  State<PantallaRegistro> createState() =>
    _PantallaRegistroState();
}

class _PantallaRegistroState
  extends State<PantallaRegistro> {
  final _formKey = GlobalKey<FormState>();
  final _nombreCtrl = TextEditingController();
  final _edadCtrl = TextEditingController();
```

```
String? _carrera;
final List<Estudiante> _estudiantes = [];

@override
void dispose() {
  _nombreCtrl.dispose();
  _edadCtrl.dispose();
  super.dispose();
}
}
```

NOTA · String? para el dropdown

`String? _carrera` admite `null` porque al inicio no hay carrera elegida. El validador del dropdown obligará a seleccionar una antes de registrar.

05 Desarrollo · Parte B — El formulario validado

3 Los campos de texto con validación

1. Dentro de un `Form`, arma los dos `TextFormField`. El de edad usa teclado numérico y valida el rango:

DART · DENTRO DEL FORM

```
TextFormField(
  controller: _nombreCtrl,
  decoration: const InputDecoration(labelText: 'Nombre'),
  validator: (v) =>
    (v == null || v.trim().length < 3)
      ? 'Minimo 3 caracteres'
      : null,
),
TextFormField(
  controller: _edadCtrl,
  keyboardType: TextInputType.number,
  decoration: const InputDecoration(labelText: 'Edad'),
  validator: (v) {
    final edad = int.tryParse(v ?? '');
    if (edad == null || edad < 15 || edad > 99) {
      return 'Edad valida: 15 a 99';
    }
    return null;
  },
),
```

4 El dropdown de carrera

1. Debajo, un `DropdownButtonFormField` que también valida que se haya elegido una carrera:

DART · DENTRO DEL FORM

```

DropdownButtonFormField<String>(
  initialValue: _carrera,
  decoration: const InputDecoration(labelText: 'Carrera'),
  items: carreras.map((c) {
    return DropdownMenuItem(value: c, child: Text(c));
  }).toList(),
  onChanged: (v) => setState(() => _carrera = v),
  validator: (v) => v == null ? 'Elige una carrera' : null,
),

```

NOTA · Un formulario, varios validadores

Cada campo lleva su propio `validator`. Al llamar a `validate()` sobre el `Form`, Flutter ejecuta todos y solo continúa si ninguno devuelve error.

5 Registrar al estudiante

1. El botón "Registrar" valida, crea el `Estudiante`, lo agrega a la lista y limpia el formulario:

```

LIB/MAIN.DART · DART
void _registrar() {
  if (!_formKey.currentState!.validate()) return;
  setState(() {
    _estudiantes.add(Estudiante(
      _nombreCtrl.text.trim(),
      int.parse(_edadCtrl.text),
      _carrera!,
    ));
    _nombreCtrl.clear();
    _edadCtrl.clear();
    _carrera = null;
  });
  _formKey.currentState!.reset();
}

```

ADVERTENCIA · `reset()` limpia los mensajes de error

`_formKey.currentState!.reset()` borra el estado visual del formulario (incluidos los errores). Combínalo con `clear()` de los controladores para dejar los campos vacíos y limpios tras cada registro.

06 Desarrollo · Parte C — Lista de estudiantes

6 Mostrar la lista con contador

1. Debajo del formulario, muestra los estudiantes registrados. Envuelve el `ListView.builder` en `Expanded` y usa `CircleAvatar` con la inicial:

DART · DENTRO DEL BUILD

```
Expanded(
  child: ListView.builder(
    itemCount: _estudiantes.length,
    itemBuilder: (context, index) {
      final e = _estudiantes[index];
      return ListTile(
        leading: CircleAvatar(
          child: Text(e.nombre[0]),
        ),
        title: Text(e.nombre),
        subtitle: Text(e.carrera + ' - ' +
          e.edad.toString() + ' años'),
        trailing: IconButton(
          icon: const Icon(Icons.delete),
          onPressed: () => setState(
            () => _estudiantes.removeAt(index),
          ),
        ),
      );
    },
  ),
);
```

2. Muestra el total en el **AppBar** con **Text** y **_estudiantes.length**. Ejecuta y registra varios estudiantes para probar validación, alta y baja.

07 Resultado esperado

Una pantalla con el formulario arriba y la lista abajo. Comportamiento esperado:

- Con el nombre vacío o menor a 3 letras, aparece error bajo el campo.
- Con edad fuera de 15–99 (o no numérica), aparece error bajo el campo.
- Sin carrera elegida, el dropdown muestra su mensaje de error.
- Al registrar válidamente, el estudiante aparece en la lista y el formulario se limpia.
- El basurero elimina al estudiante y el contador del AppBar se actualiza.

08 Verificación de resultados y entregable

Marca cada punto como verificación de que completaste el ejemplo:

- ☐ El modelo Estudiante agrupa nombre, edad y carrera.
- ☐ Los tres campos validan según sus reglas.
- ☐ Solo se registra cuando el formulario completo es válido.
- ☐ Tras registrar, el formulario se limpia (clear + reset).
- ☐ La lista muestra a los estudiantes y permite borrarlos.
- ☐ El AppBar muestra el total de estudiantes.

EJEMPLO · Qué entregar

El archivo `lib/main.dart` terminado y un documento (PDF o Word) con: (1) captura de un error de validación; (2) captura de la lista con al menos tres estudiantes y el contador; y (3) **reto personal:** agrega un campo más (por ejemplo, `semestre` con validación 1–10) y muéstralo en el `subtitle` del `ListTile`. Descríbelo en 3–5 líneas.

09 Cuestionario

1. ¿Por qué la variable de la carrera se declara como String? (con signo de interrogación)?
2. ¿Qué diferencia hay entre validar un solo campo y llamar a `validate()` sobre todo el Form?
3. ¿Para qué sirve `reset()` del `FormState` y en qué se diferencia de `clear()` del controlador?
4. ¿Por qué se usa `int.tryParse` en el validator pero `int.parse` al crear el Estudiante?
5. Explica cómo el contador del `AppBar` se mantiene sincronizado con la lista.

10 Rúbrica de evaluación

Criterio	Puntaje
Modelo Estudiante y lista de carreras	15 %
Formulario con validación de los 3 campos	35 %
Alta, limpieza (<code>clear+reset</code>) y baja de la lista	25 %
Presentación de la lista y contador	10 %
Cuestionario resuelto	15 %

11 Referencias

- Flutter — Build a form with validation. docs.flutter.dev/cookbook/forms/validation
- Flutter — DropdownButtonFormField (API). [api.flutter.dev · DropdownButtonFormField](https://api.flutter.dev/DropdownButtonFormField)
- Flutter — Work with long lists (`ListView.builder`). docs.flutter.dev/cookbook/lists/long-lists

Ejemplo integrador de las Guías 1–5. Contenido basado en la documentación oficial de Flutter (v3.44). Elaborado para INF 662 — UATF.